

252-0027

**Einführung in die Programmierung
Übungen**

Woche 3: Einführung Java

Jhalak Yadav

Departement Informatik

ETH Zürich

Organisatorisches

- Mein Name: Jhalak Yadav
- Bei Fragen: jyadav@student.ethz.ch
 - Mails bitte mit «[EProg26]» im Betreff
- Neue Aufgaben: **Dienstag Abend** (im Normalfall)
- Abgabe der Übungen bis **Dienstag Abend (23:59)** Folgewoche
 - Abgabe immer via Git
 - Lösungen in separatem Projekt auf Git

Theorie

Auswertung – Arithmetische Operatoren

Operator	Beschreibung	Kurzbeispiel
+	Addition	<code>int</code> antwort = 40 + 2;
-	Subtraktion	<code>int</code> antwort = 48 - 6;
*	Multiplikation	<code>int</code> antwort = 2 * 21;
/	Division	<code>int</code> antwort = 84 / 2;
%	Teilerrest, Modulo-Operation, errechnet den Rest einer Division	<code>int</code> antwort = 99 % 57;
+	positives Vorzeichen	<code>int</code> j = +3;
-	negatives Vorzeichen	<code>int</code> minusJ = -j;

Assoziativität

- Die **Assoziativität** («**associativity**») eines Operators bestimmt, wie ein Operand verwendet wird
- Ist ein Operator $\otimes$
 - **Linksassoziativ** («**left-associative**») $\rightarrow X \otimes Y \otimes Z$ gleich $(X \otimes Y) \otimes Z$
 - **Rechtsassoziativ** («**right-associative**») $\rightarrow X \otimes Y \otimes Z$ gleich $X \otimes (Y \otimes Z)$
- Die meisten Operatoren in Java sind linksassoziativ (Ausnahmen: Assignment, Ternary und Unary Operatoren)

Operator-Präzedenz

Höhere
Präzedenz



postfix

expr++ expr--

unary

++expr --expr +expr -expr ~ !

multiplicative

* / %

additive

+ -

relational

< > <= >= instanceof

equality

== !=

logical AND

&&

logical OR

||

ternary

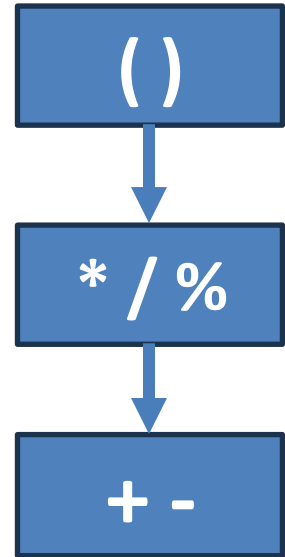
? :

assignment

= += -= *= /= %= &= ^= |= <<= >>= >>>=

Auswertung – Ordnung

- Operand wird vom Operator mit höherer Rangordnung («precedence») verwendet
- Wenn zwei Operatoren dieselbe Rangordnung haben, dann entscheidet die Assoziativität
- Wenn etwas anderes gewünscht wird: Klammern verwenden!



Auswertung - Typ

- Gleiche Operanden = Klarer Typ

int + int = int

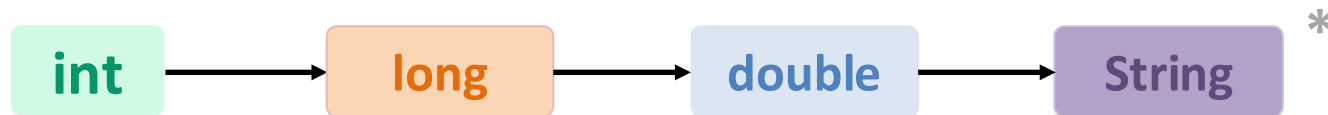
double + double = double

- Was passiert aber, wenn:

double + int = ?

Implizites Casten

- Generelle Regel:



- Beispiele:



* Dies ist eine Vereinfachung: Weitere Typen und genaues casten werden später genauer betrachtet

Kurzschlussauswertung («short-circuit evaluation»)

Bei Berechnung von `p && q` und `p || q`

- Java wertet zuerst den linken Operanden (`p`) und dann den rechten (`q`) aus
- die Auswertung wird beendet, sobald das Ergebnis feststeht.
 - Bei `p && q`: Falls `p == false`, dann ist `p && q == false`
 - Bei `p || q`: Falls `p == true`, dann ist `p || q == true`

p	q	p && q
true	true	true
true	false	false
false	true	false
false	false	false

p	q	p q
true	true	true
true	false	true
false	true	true
false	false	false

In diesen Fällen ist keine Auswertung von `q` nötig!

Ersatzdarstellungen («escape sequences»)

Geben uns die Möglichkeit Sonderzeichen auszudrücken

- Fangen alle mit \ (Rückwärtsschrägstrich, «backslash») an
- Beispiel aus der Vorlesung:

```
"\\Hallo\\nWie\\tgeht's \\\"uns\\\"?\\\"\\\""
```

- Ausgabe:

```
\\Hallo  
Wie      geht's  \"uns\"?\\
```

Escape Sequences

Escape Sequence	Description
\\t	Insert a tab in the text at this point.
\\b	Insert a backspace in the text at this point.
\\n	Insert a newline in the text at this point.
\\r	Insert a carriage return in the text at this point.
\\f	Insert a form feed in the text at this point.
\\'	Insert a single quote character in the text at this point.
\\"	Insert a double quote character in the text at this point.
\\\\	Insert a backslash character in the text at this point.

Auswertung – Beispiele

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(2/3+4.5/3);  
4     }  
5 }
```

Output: 1.5

Auswertung – Beispiele

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(8%3+2.5+4/3);  
4     }  
5 }
```

Output: 5.5

Auswertung – Casting

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println((double)5/2);  
4     }  
5 }
```

Output: 2.5

Auswertung – String

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Wir schreiben das Jahr "+2023/2*2.0);  
4     }  
5 }  
6
```

Output: Wir schreiben das Jahr 2022.0

Auswertung – Prüfungsbeispiel

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(12/4*3+("X"+7%3)+18%5*2);  
4     }  
5 }
```

Output: 9X16

Auswertung – Prüfungsbeispiel

()

* / %

+ -

```
1
2 public class Main {
3     public static void main(String[] args) {
4         System.out.println("Wir schreiben das Jahr " + 2000 + 24);
5     }
6 }
```

Output: Wir schreiben das Jahr 200024

Aufgabe 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```

760

Aufgabe 2



```
1  class MyClass{
2      public static void main(String[] args){
3          System.out.println(8 / 5 + 0.5 + 5 / 2 + (8 % 3) * 1.0);
4      }
5  }
```

5.5

Rechnung - Prüfungsaufgabe

```
1. System.out.println(36 / (3 * 12    4));
```

```
//Output: 4
```

```
2. System.out.println(64 % (4    8 + 1) / 4);
```

```
//Output: 7
```

```
3. System.out.println(32    (2    6));
```

```
//Output -8
```

Bestimmen Sie für jede Anweisung die fehlenden Operatoren so, das die Anweisung die gezeigte Ausgabe erzeugt. Mögliche Operatoren sind +, -, *, / und %.

Rechnung - Prüfungsaufgabe

1. `System.out.println(36 / (3 * 12 / 4));`

`//Output: 4`

2. `System.out.println(64 % (4 * 8 + 1) / 4);`

`//Output: 7`

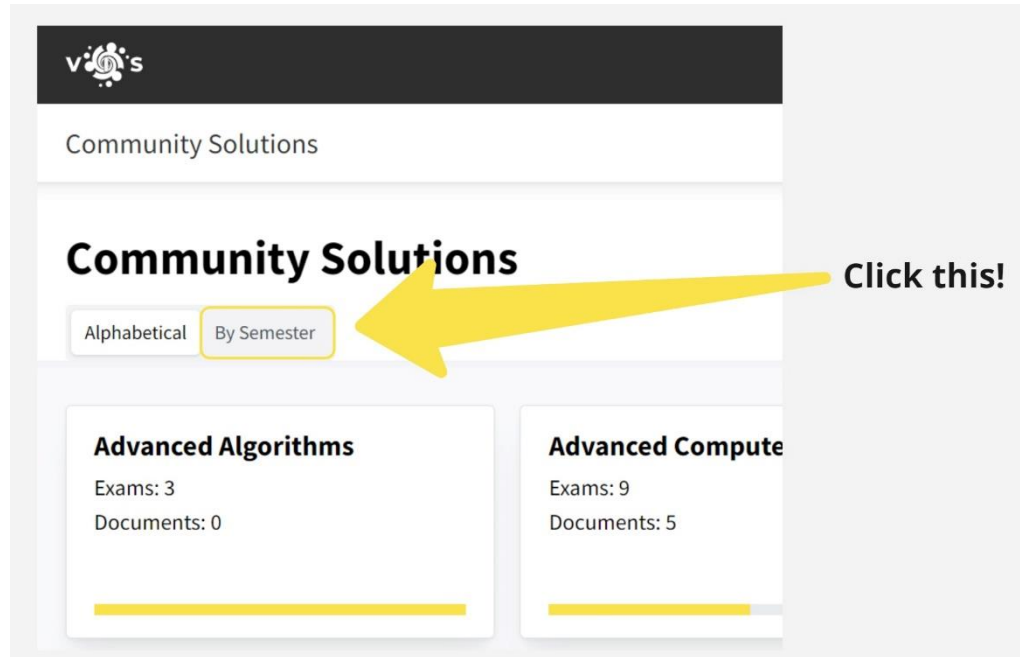
3. `System.out.println(32 / (2 - 6));`

`//Output -8`

Bestimmen Sie für jede Anweisung die fehlenden Operatoren so, das die Anweisung die gezeigte Ausgabe erzeugt. Mögliche Operatoren sind +, -, *, / und %.

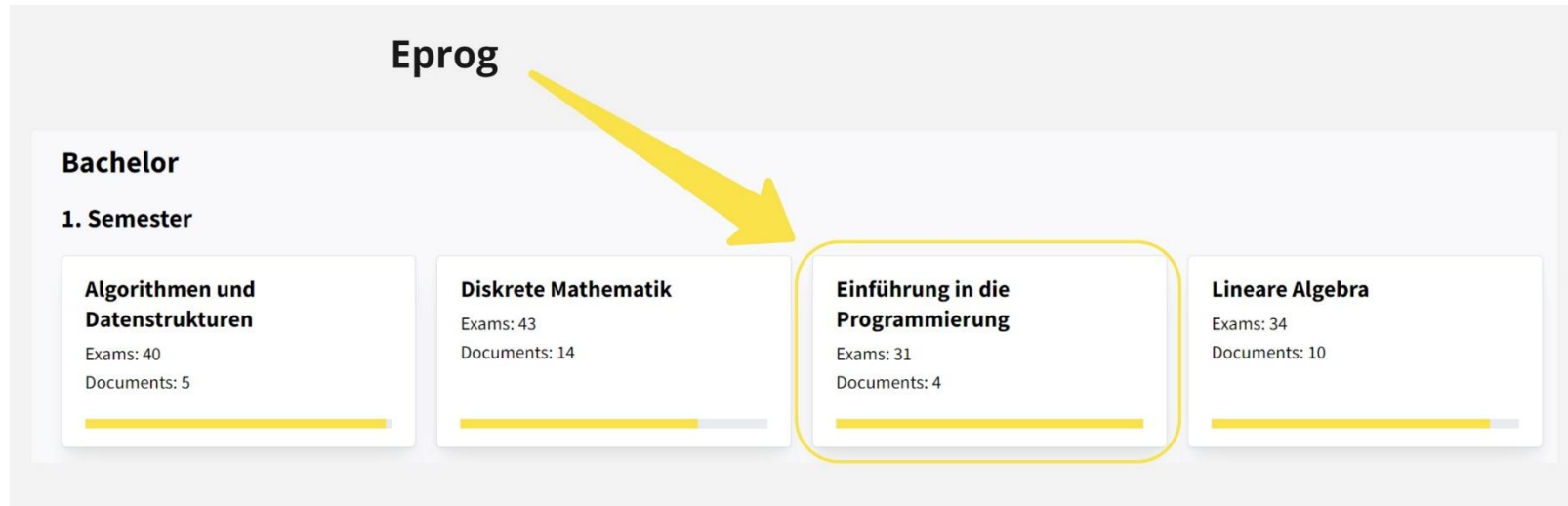
Rechnung - Prüfungsaufgabe

- Prüfungen auf: <https://exams.vis.ethz.ch>



Rechnung - Prüfungsaufgabe

- Prüfungen auf: <https://exams.vis.ethz.ch>



Rechnung - Prüfungsaufgabe

Wenn es für eine Anweisung mehrere mögliche Lösungen gibt, so genügt eine Lösung. Sollte es keine Lösung geben, so schreiben Sie bitte "nicht möglich".

Bereits existierende Operatoren (oder sonstige Teile der Anweisung) dürfen Sie *nicht* verändern. Auch dürfen Sie Klammern weder hinzufügen noch entfernen.

```
1. System.out.println(36 / (3 * 12    4));
```

```
//Output: 4
```

Show Answers

Click this!

Rechnung - Prüfungsaufgabe

- Alte Prüfungen auf: <https://exams.vis.ethz.ch>
- Alte Prüfungen lösen ist nicht Teil des Semesters.
 - Dafür bleibt genügend Zeit in der Lernphase in der vorlesungsfreien Zeit vor den Prüfungen.
- Alte Prüfungsaufgaben werden von Zeit zu Zeit in den Übungen kommen, aber es verbleiben genügend alte Prüfungen, welche ihr selbstständig lösen könnt.

Kahoot

Nachbesprechung Übung 2

Weitere Fragen zu IntelliJ oder Git?

(Interactive)

Lösung Übung 2, Aufgabe 2

```
/*  
 * Author: Maximiliana Muster  
 * für Einführung in die Programmierung  
 */  
public class HelloProgrammer {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, my name is Max!");  
    }  
}
```

Lösung Übung 2, Aufgabe 4

1. Erstellen Sie eine Beschreibung `<geradezahl>`, die als legale Symbole alle geraden Zahlen (d.h. Zahlen, die ohne Rest durch 2 teilbar sind) zulässt. Beispiele sind +02, 4, 10, -20.

Lösung Übung 2, Aufgabe 4

2. Zeigen Sie in einer Tabelle, dass Ihre Beschreibung das Symbol “28” als gerade Zahl erkennt.

Lösung Übung 2, Aufgabe 4

3. Erstellen Sie eine Beschreibung `<x2ygemischt>`, die als legale Symbole genau jene Wörter zulässt, in denen für jedes "X" zwei "Y" als Paar auftreten. Beispiele sind `XYX`, `YYX`, `XYYYXX`, `XXYYYY`. Die Beschreibung schließt den leeren String nicht aus.

Lösung Übung 2, Aufgabe 4

Die folgenden EBNF-Beschreibungen sind nicht äquivalent. Finden Sie ein *kürzestmögliches* Symbol, das von der einen Beschreibung als legal erkannt wird, aber nicht von der anderen. (Fangen Sie mit einfachen Kombinationen von A und B an.)

EBNF-Beschreibung: <beispiel1>

<beispiel1> $\Leftarrow$ [A][B]

EBNF-Beschreibung: <beispiel2>

<beispiel2> $\Leftarrow$ [A[B]]

Lösung Übung 2, Aufgabe 4

5. Erstellen Sie eine EBNF Beschreibung `<doppelt>`, die als legale Symbole genau jene Wörter zulässt, in denen die doppelte Anzahl "Y" nach einer Folge von "X" auftritt. Beispiele sind `XYX`, `XXYYYYX`, usw. Die Beschreibung schliesst den leeren String nicht aus.

Vorbesprechung Übung 3

Ranking



- **Prüfungsrelevant**



- **Empfehlenswert**



- **Nicht prüfungsrelevant**

Aufgabe 1: Fehlerbehebung



1. Öffnen Sie die Datei `FollerVehler.java` im Ordner `u03` → `FollerVehler` → `src` → `main` → `java`.
2. Versuchen Sie, das Programm auszuführen. IntelliJ zeigt Ihnen die Fehlermeldungen an.
3. Versuchen Sie zunächst, die Fehler durch reines Lesen des Codes zu finden und zu überlegen, wie man sie beheben könnte. (Tipp: Achten Sie auf Klammern, Gross-/Kleinschreibung, Semikolons und Methodennamen.)
4. Beheben Sie die Fehler schrittweise, bis sich `FollerVehler.java` fehlerfrei kompilieren und ausführen lässt. IntelliJ zeigt Ihnen dabei weiterhin die Fehlermeldungen an und unterstützt Sie beim Beheben der Probleme.

Wenn alle Fehler behoben sind, sollte das Programm Folgendes ausgeben:

```
Hello world
```

```
Gefällt Ihnen dieses Programm?
```

```
Ich habe es selbst geschrieben.
```

Aufgabe 1: Fehlerbehebung



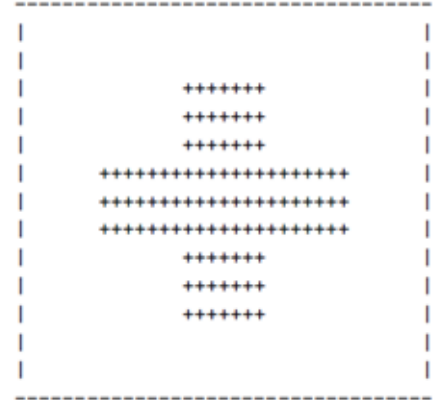
- In Java werden **alle** Java-Dateien in einem Projekt kompiliert.
- Wenn auch **nur eine** Java-Datei nicht kompiliert, dann kann **kein** Java-Code ausgeführt werden in dem Projekt.
- Deshalb benutzen wir Subprojekte, damit Aufgaben 2-4 kompilieren, auch wenn Aufgabe 1 nicht kompiliert.

Aufgabe 2: Schweizerfahne



Statt eine Vorlage zu benützen, schreiben Sie in dieser Aufgabe ein Programm von Grund auf selbst. IntelliJ wird Ihnen allerdings etwas Schreibarbeit abnehmen.

1. Erstellen Sie eine neue Java-Datei "SwissFlag.java". Rechts-Klicken Sie dafür den *src*-Ordner und wählen Sie *New* → *Java Class*. Geben Sie den Namen der Klasse ein ("SwissFlag.java") und bestätigen Sie mit der Enter-Taste. Danach wählen Sie *Add*, wenn Sie gefragt werden, ob Sie die Datei zu git hinzufügen möchten.
2. Erstellen Sie zuerst eine leere *main*-Methode, so wie Sie es bei anderen Programmen gesehen haben.
3. Erweitern Sie das Programm so, dass es die Schweizerfahne in der Konsole ausgibt. Die Fahne könnte ungefähr wie folgt aussehen, Sie dürfen aber auch eine grössere oder schönere Version entwerfen:



Teilen Sie das Programm in mehrere Methoden auf, welche von der *main*-Methode aufgerufen werden. Damit sorgen Sie dafür, dass weniger Wiederholungen von Code-Stücken vorkommen, was das Ändern des Programms deutlich einfacher macht.

Aufgabe 3: Eingabe und Zufall



In dieser Aufgabe arbeiten Sie mit der Eingabe und Ausgabe von Java und lernen die Klassen `Scanner` und `Random` näher kennen.

1. Schreiben Sie ein Programm “`Adder.java`”, welches zwei ganze Zahlen einliest und die Summe davon ausgibt. Sie sollen dafür die `Scanner`-Klasse verwenden, wie in der Vorlesung gezeigt. Das Programm soll nach der ersten Zahl fragen:

Geben Sie Zahl 1 ein:

dann nach der zweiten Zahl:

Geben Sie Zahl 2 ein:

und schliesslich, wenn Sie zum Beispiel “4” und “1999” eingeben, folgendes ausgeben:

$4 + 1999 = 2003$

Sie können davon ausgehen, dass nur ganze Zahlen eingegeben werden. Testen Sie das Programm mit verschiedenen Zahlen.

2. Schreiben Sie ein Programm `Wuerfel.java`, das einen Würfel simuliert. Hierbei soll eine positive ganze Zahl `N` eingelesen werden, welche die Anzahl der Seiten des Würfels repräsentiert. Der übliche Würfel hat 6 Seiten, jedoch existieren auch Würfel mit 12, 16, 20 (siehe Abbildung 1) und mehr Seiten. Jede Seite trägt eine unterschiedliche Zahl, die von 1 bis `N` (inklusive `N`) reicht.

Das Programm soll den Wurf simulieren, indem es die Klasse `Random` verwendet. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Wie viele Seiten hat Ihr Würfel?

Der Benutzer gibt eine Zahl ein, z.B. 20, danach wird gewürfelt:

Es wurde eine 17 gewürfelt!



3. Schreiben Sie ein Programm `ChatGTP.java`, welches die Interaktion mit einem AI-Chatbot simuliert. Das Programm soll den Benutzer begrüßen, Sie nach Ihrem Namen und Alter fragen, und dem Benutzer Ihre Glückszahl verraten. Die Glückszahl soll zufällig gewählt werden, in dem Sie die `Random` Klasse benutzen. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Guten Tag! Ich bin ChatGTP, der beste Chatbot, denn es gibt! Wie heissen Sie?

Ein Namen wird eingegeben, z.B. Alice, danach antwortet der Chatbot auf diesen Input:

Sehr erfreut Alice! Wie alt sind Sie?

Ein Alter wird eingegeben, z.B. 23, danach antwortet der Chatbot auf diesen Input:

Mittels dieser Informationen habe ich Ihre Glückszahl gefunden! Die Glückszahl lautet 42.

Aufgabe 4: Berechnung



1. Vervollständigen Sie “SharedDigit.java”. In der main-Methode sind zwei int-Variablen a und b deklariert und mit einem Wert zwischen 10 und 99 (einschliesslich) initialisiert. Das Programm soll einer int-Variablen r einen bestimmten Wert zuweisen. Wenn a und b eine Ziffer gemeinsam haben, dann wird r die gemeinsame Ziffer zugewiesen (wenn a und b beide Ziffern gemeinsam haben, dann kann eine beliebige Ziffer zugewiesen werden). Wenn es keine gemeinsame Ziffer gibt, dann soll -1 zugewiesen werden. Sie brauchen für dieses Programm keine Schleife.

Beispiele:

- Wenn a: 34 und b: 53, dann ist r: 3
- Wenn a: 10 und b: 22, dann ist r: -1
- Wenn a: 66 und b: 66, dann ist r: 6
- Wenn a: 34 und b: 34, dann ist r: 3 oder 4

Testen Sie Ihre Lösung mit a gleich 34 und b gleich 43. Was liefert Ihr Programm?

2. Vervollständigen Sie "SumPattern.java". In der main-Methode sind drei int-Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Wenn die Summe von zwei der Variablen die dritte ergibt, nehmen wir an dass $a + c == b$, so soll die Methode "Moeglich. $a + c == b$ " ausgeben (wobei die Werte für a, b, und c einzusetzen sind). Wenn das nicht der Fall ist, dann soll die Methode "Unmoeglich." ausgeben.

Beispiele:

- Wenn a: 4, b: 10, c: 6, dann wird "Moeglich. $4 + 6 == 10$ " oder "Moeglich. $6 + 4 == 10$ " ausgegeben.
 - Wenn a: 2, b: 12, c: 0, dann wird "Unmoeglich." ausgegeben.
3. Vervollständigen Sie "AbsoluteMax.java". In der main-Methode sind drei int-Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Das Programm soll einer int-Variable r den grössten absoluten Wert von a, b, und c zuweisen.

Aufgabe 5: EBNF



Hinweis: Diese EBNF-Aufgabe ist auch als [Moodle-Quiz](#) verfügbar. Das Moodle-Quiz enthält in einigen Teilaufgaben zusätzliche Übungen und ist daher auch für Studierende hilfreich, die nur die Papierversion bearbeiten möchten.

In dieser Aufgabe erstellen Sie erneut verschiedene EBNF-Beschreibungen. Speichern Sie diese wie gewohnt in der Text-Datei “EBNF.txt”, welche sich im Ordner “u03” → “resources” befindet. Sie können die Datei direkt in IntelliJ bearbeiten.

1. Erstellen Sie eine Beschreibung `<pyramid>`, welche als legale Symbole genau jene Wörter zulässt, welche aus einer Folge von strikt aufsteigenden, gefolgt von einer Folge von strikt absteigenden Ziffern, bestehen. Beispiele sind: 14, 121, 1221, 1341.
Sie dürfen annehmen, dass das leere Wort auch zugelassen wird. (Als Challenge können Sie probieren, das leere Wort auszuschliessen.)

Aufgabe 5: EBNF



2. Erstellen Sie eine Beschreibung $\langle \text{digitsum} \rangle$, welche als legale Symbole genau jene natürlichen Zahlen zulässt, deren Quersumme eine gerade Zahl ist.
3. Erstellen Sie eine Beschreibung $\langle \text{xyz} \rangle$, welche genau alle Wörter zulässt, die aus X, Y und Z bestehen und bei welchen jedes X mindestens ein Y im Teilwort links und rechts von sich hat. Beispiele sind: Z, YXY, YXXY, ZYXYY.
4. Erstellen Sie eine Beschreibung $\langle \text{term} \rangle$, welche als legale Symbole genau alle wohlgeformten arithmetischen Terme, bestehend aus positiven ganzen Zahlen, Variablen (x, y, z), Addition und Klammern zulässt. Geklammerte Terme müssen mindestens eine Addition enthalten. Beispiele sind: $1 + 4$, $(1 + 4)$, $1 + (3 + 4)$, $(1 + 1) + x + 5$.